

Projekt DevOps

“Development aplikacji web za pomocą k8s&helm”

Mikalay Tsepliakouski

Ogólny pomysł/koncepcja

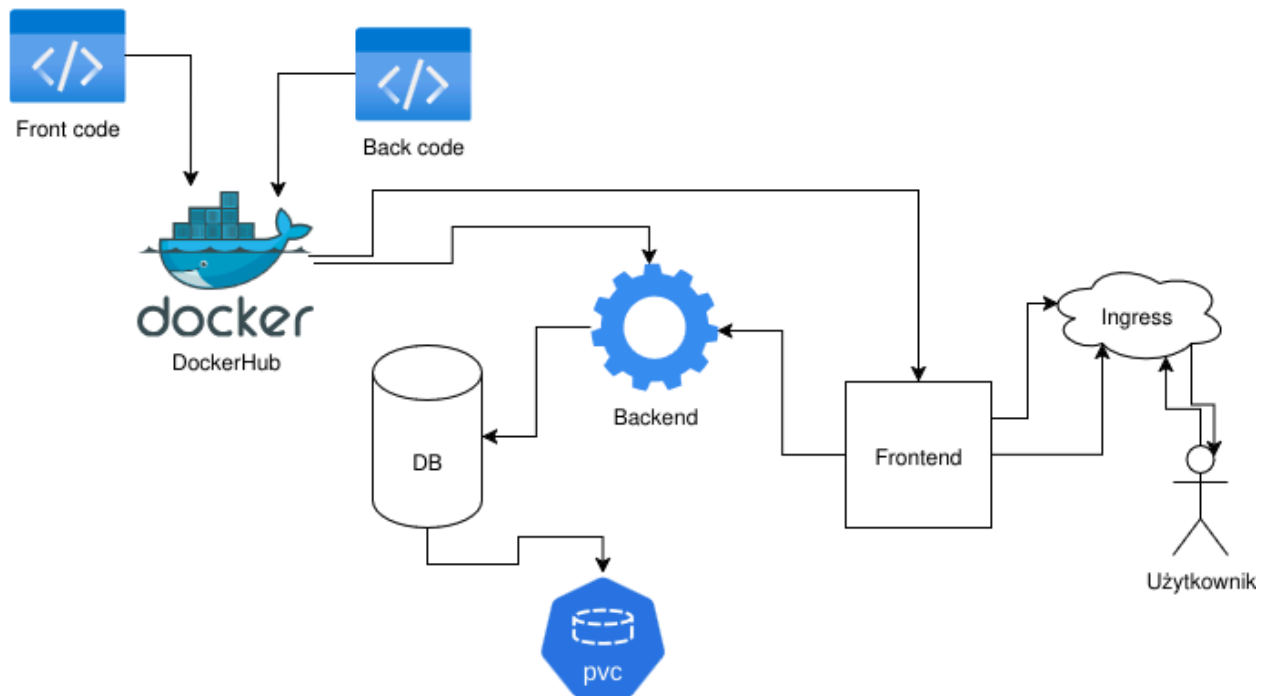
Ponieważ projekt miał stanowić podsumowanie kursu *DevOps*, naturalnym wyborem było wdrożenie aplikacji webowej. Aby nadać mu bardziej realistyczny charakter i lepiej wcielić się w rolę DevOpsa, zdecydowałem się na wdrożenie rzeczywistej aplikacji webowej stworzonej przez mojego kolegę:

<https://github.com/ShadowCatP/job-applications-tracker/tree/main>

Komponenty web aplikacji:

- Frontend – aplikacja webowa napisana w Next.js
- Backend – serwer REST API napisany w Node.js z użyciem Express
- Baza danych – MongoDB

Po zbadaniu działania wybranej aplikacji oraz znalezieniu potrzebnych elementów środowiska (bo dokumentacji nie było xd) zdecydowałem się na następujący schemat:



- Kod backendu i frontendu jest umiejscowiony w odpowiednich direktoriach z których za pomocą Dockerfile zostają zbudowane obrazy ich kontenerów.
- Obrazy te zostają "wypychane" do dockerhub
- Obrazy z dockerhub zostają użyte do tworzenia podów
- Aby zapewnić trwałość danych, baza danych używa PVC
- Komunikacja z frontendem odbywa się przez ingress, co zapewnia wygodną separację "/" i "/api" + po prostu lepiej wygląda niż node port.

- **Ważne:** schemat jest przykładowy i nie odzwierciedla pełnej struktury aplikacji (np. "backend" na schemacie to w rzeczywistości deployment + sekret + configMap+service)
- **Ważne:** w przypadku strzałek między elementami aplikacji, pokazana została typowa strategia ruchu "frontend może inicjalizować połączenia z backendem, backend przyjmować z frontend i "mówić" do bazy a baza nie może nawiązywać połączeń".
Polityki ruchu zostały określone w **netpols.yaml**.

Helm chart

```
namespace:
  namespaceSeparation: true
  name: job-appl-namespace

quota:
  pods: "6"
  services: "3"
  secrets: "1"
  configmaps: "1"
  persistentvolumeclaims: "2"

requests.cpu: "3"
requests.memory: "4Gi"
limits.cpu: "4"
limits.memory: "8Gi"
```

```
{{- if .Values.namespace.namespaceSeparation }}
namespace: {{.Values.namespace.name}}
{{- end}}
```

Domyślnie chart instaluje aplikację w osobnym namespace, dla którego zdefiniowane są limity zasobów. Jeśli jednak nie chcemy korzystać z oddzielnego namespace, można to wyłączyć, ustawiając parametr **namespaceSeparation** na false.

Frontend

Image na bazie **node:24**.

```
service:
  type: ClusterIP
  port: 80
  targetPort: 3000
```

Porty: 3000 kontener, przez 80 service.

Environment:

APIbase url oraz **config.json** dla nginx są w config map, a potem zmienna normalnie jak zmienna a plik się montuje:

```

env:
  {{- range $key, $value := .Values.front.env }}
  - name: {{ $key }}
    valueFrom:
      configMapKeyRef:
        name: {{ .Values.front.name }}-confmap
        key: {{ $key }}
  {{- end }}

volumeMounts:
  - name: frontend-config
    mountPath: /usr/share/nginx/html/config.json
    subPath: config.json

```

Dla tego, jak i dla wszystkich innych elementów aplikacji określono limity zasobów:

```

{{- with .Values.front.resources }}
resources:
  {{- toYaml . | nindent 12 }}
{{- end }}

```

```

resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "1"

```

Backend

Też kontener na bazie **node:24**.

Porty: 5000 i w kontenerze i przez service.

Environment:

Przyjmuje 2 zmienne środowiskowe:

- PORT
- MONGO_URI – uri aby łączyć się z bazą danych mongodb

Zmienne te normalnie są podawane przez configMap

Backend także potrzebuje JWT secret, który jest dodawany do secret.yaml i potem też do deployment:

```

envFrom:
  - configMapRef:
    name: {{ .Values.back.name }}-confmap
  - secretRef:
    name: {{ .Values.back.name }}-secret

```

Ponieważ backend zazwyczaj potrzebuje trochę czasu by się uruchomić, dla niego zrobiono readiness i liveness probe'y:

- liveness polega na "pingowaniu" **/healthz**

```
app.get('/healthz', (req, res) => {  
  res.status(200).send('OK');  
});
```

```
livenessProbe:  
  httpGet:  
    path: /healthz  
    port: {{ .Values.back.containerPort }}  
  initialDelaySeconds: 10  
  periodSeconds: 20  
  timeoutSeconds: 2
```

- Readiness sprawdza czy baza danych jest już osiągalna, pytając o **readyState**

```
// Readiness probe which will check the db connection  
app.get('/ready', (req, res) => {  
  const state = mongoose.connection.readyState;  
  if (state === 1) {  
    res.status(200).send('OK');  
  } else {  
    res.status(503).send('Not Ready');  
  }  
});
```

```
# is accessible overall  
readinessProbe:  
  httpGet:  
    path: /ready  
    port: {{ .Values.back.containerPort }}  
  initialDelaySeconds: 5  
  periodSeconds: 10  
  timeoutSeconds: 2
```

Baza danych

Image to zwykły **mongo:6**

Porty: 27017 i w kontenerze i przez service.

Jedynie co warto dodać w kontekście bazy danych to to, że utworzony został PVC, aby zapewnić trwałość danych.

```
spec:  
  accessModes:  
    - {{ .Values.db.storage.accessMode }}  
  resources:  
    requests:  
      storage: {{ .Values.db.storage.size }}  
    volumeMode: {{ .Values.db.storage.volumeMode }}  
    storageClassName: {{ .Values.db.storage.className }}
```

```
  volumeMounts:  
    - name: mongo-storage  
      mountPath: {{ .Values.db.storage.mountPath }}  
  volumes:  
    - name: mongo-storage  
      persistentVolumeClaim:  
        claimName: {{ .Values.db.name }}-pvc
```

Dodatkowe elementy

Wdrożone networkPolicy ograniczają ruch pomiędzy komponentami aplikacji do minimalnych niezbędnych ścieżek komunikacyjnych:

- Frontend — tylko egress w kierunku backendu.
- Backend — ingress wyłącznie z frontendu, oraz egress do bazy danych
- Baza danych — ingress wyłącznie z backendu
- Dodatkowo wszystkie elementy mają dostęp do DNS

W rezultacie jedyną dozwoloną ścieżką komunikacji jest: frontend → backend → baza + dostęp do DNS.

Jak wspomniano wcześniej, jeżeli chart tworzy się w osobnym namespace, dla tego namespace zostaje określona resource Quota.

```
{{- if .Values.namespace.namespaceSeparation }}
apiVersion: v1
kind: ResourceQuota
metadata:
  name: {{ include "job-appl-chart.name" . }}-quota
  namespace: {{ .Values.namespace.name }}
spec:
  hard:
    {{- with .Values.quota }}
    | {{- toYaml . | nindent 4 }}
    {{- end }}
```

Perspektywy rozwoju

Na koniec chciałbym szybko omówić rzeczy które warto by było dodać, w przypadku dalszego rozwoju projektu, zaangażowania większej ilości developerów (lub gdy znajdę więcej czasu :))

Dot. kubernetesa:

- RBAC
- Skalowanie poziome
- imagePullPolicy i odpowiednio określić strategię wdrażania nowych podów (czyli strategię w deployment)
- więcej health checks

Inne rzeczy do ulepszenia workflow:

- NA PEWNO hooks z linterami
- GitHub (czy inne) actions DevOps/DevSecOps pipeline z przeglądem jak statycznie (SAST) kodu aplikacji, tak i dynamicznej analizy (np. DAST z OWASP)